



테스트 주도 개발방법론

박대우

수업 개요

강의 계획

주차	일자	강의 주제	강의 세부 내역
1 주차	2026-03-07	- 강의 Overview - TDD 개념	
2 주차	2026-03-14	소프트웨어 테스트 개념	
3 주차	2026-03-21	- 클린코드 - 클린코드를 위한 개발환경	
4 주차	2026-03-28	웹프런트 엔드 기초 & 부트스트랩	
5 주차	2026-04-04	Django 프로젝트 앱 개발하기	
6 주차	2026-04-11	정적파일과 미디어 파일 관리하기	
7 주차	2026-04-18	TDD 관련 사례 조사 발표	
8 주차	2026-04-25	중간 시험(또는 대체 과제)	

수업 개요

강의 계획

주차	일자	강의 주제	강의 세부 내역
9 주차	2026-05-02	▪ TDD로 템플릿 모듈화	
10 주차	2026-05-09	▪ 다대일 관계 구현	
11 주차	2026-05-16	▪ 다대다 관계 구현	
12 주차	2026-05-23	▪ Form으로 포스트 작성	
13 주차	2026-05-30	▪ 외부 라이브러리 활용	
14 주차	2026-06-06	▪ 대문페이지와 자기소개페이지 완성	
15 주차	2026-06-13	▪ 프로젝트 발표	

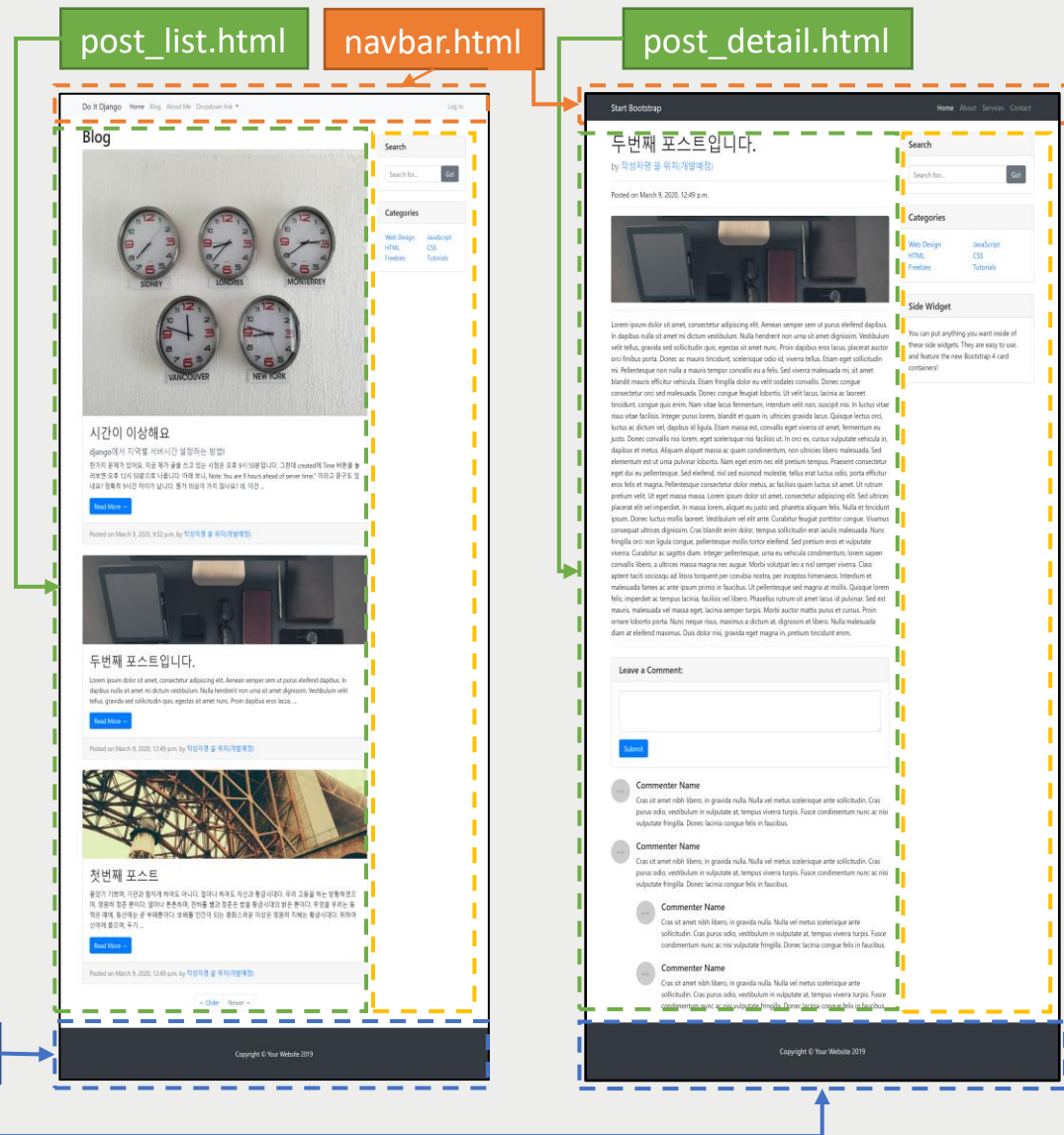
다대일 관계 구현

- 템플릿 모듈화
- 다대일 관계 구현

1. 템플릿을 모듈화 하는 이유

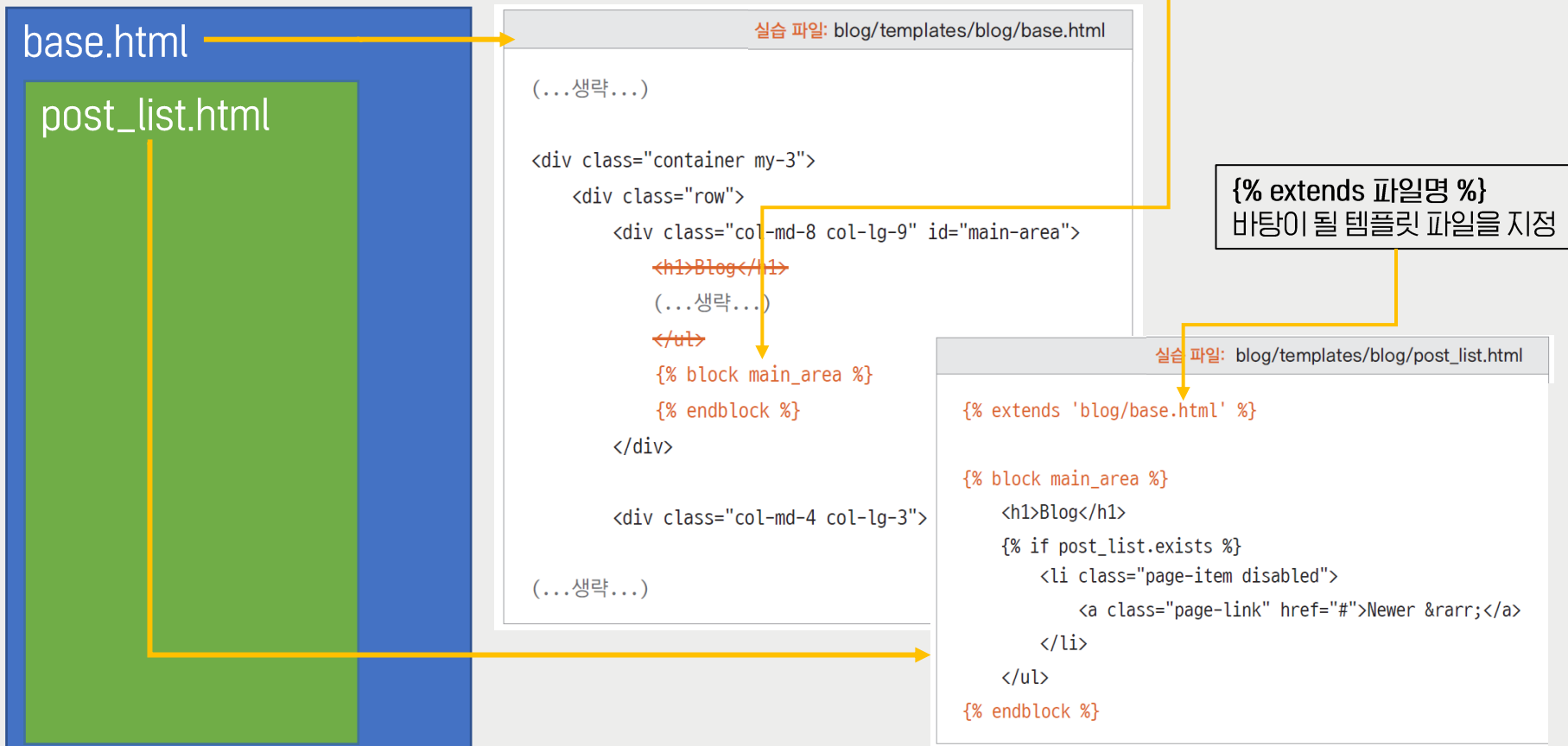
□ 웹사이트의 일관적인 디자인 적용

- ✓ 코드 재사용성 ↑
- ✓ 수정사항 반영 쉬움



2. 메인영역 모듈화하기

❑ block과 extends



2. 메인영역 모듈화하기

❑ post_list.html 모듈화하기

✓ base.html 만들기(post_list.html 복사해서): 공통영역만 남기기 위함

- id="main-area" 인 div 요소 내부 모두 삭제
- {% block main_area %}와 {% endblock %} 추가

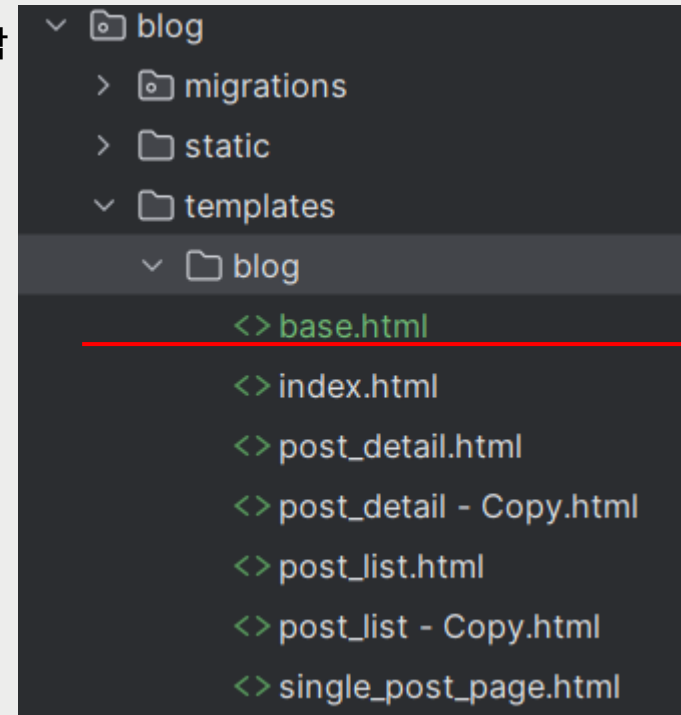
실습 파일: blog/templates/blog/base.html

```
(...생략...)

<div class="container my-3">
  <div class="row">
    <div class="col-md-8 col-lg-9" id="main-area">
      <h1>Blog</h1>
      (...생략...)
      </ut>
      {% block main_area %}
      {% endblock %}
    </div>

    <div class="col-md-4 col-lg-3">

      (...생략...)
```



2. 메인영역 모듈화하기

□ post_list.html 모듈화하기

✓ base.html를 확장해 post_list.html에 넣기

- base.html에서 지웠던 부분만 남기고 나머지 삭제
- {% extends 'blog/base.html' %}
- {% block main_area %}, {% endblock %} 블록의 시작과 끝

✓ **테스트 코드와 웹브라우저에서 결과확인:** python manage.py test blog.tests.TestView.test_post_list

```

실습 파일: blog/templates/blog/post_list.html

{% extends 'blog/base.html' %}

{% block main_area %}
    <h1>Blog</h1>
    {% if post_list.exists %}
        . . . . .
        <li class="page-item disabled">
            <a class="page-link" href="#">Newer &rarr;</a>
        </li>
    </ul>
{% endblock %}
    
```


2. 메인영역 모듈화하기

❑ post_detail.html 모듈화하기

✓ base.html를 확장해 post_detail.html 에 넣기

- 메인영역에 해당하는 부분만 남기고 나머지 삭제
- <div class="container">안에 <div class="row">가 있고, <div class="col-lg-8">과 <div class="col-md-4">로 8:4로 나뉜 구조. <div class="col-lg-8">만 남김

post_detail.html

```

1  {% extends 'blog/base.html' %}
2
3  {% block main_area %}
4
5      <!-- Title -->
6      <h1 class="mt-4">{{ post.title }}</h1>
7      <h5 class="text_muted">{{ post.hook_text }}</h5>
8
9      <!-- Author -->
10     <p class="lead">

```

```

52     <!-- Comments Form -->
53     <div class="card my-4">...>
64
65     <!-- Single Comment -->
66     <div class="media mb-4">...>
73
74     <!-- Comment with nested comments -->
75     <div class="media mb-4">...>
99
100    {% endblock %}

```

2. 메인영역 모듈화하기

□ post_detail.html 모듈화하기

✓ 제목 블록 따로 만들기

- 테스트 수행: 웹브라우저 위쪽에 나타나는 타이틀 없음

```
=====
FAIL: test_post_detail (blog.tests.TestView.test_post_detail)
-----
Traceback (most recent call last):
  File "C:\work\python\mysite\blog\tests.py", line 72, in test_post_detail
    self.assertIn(post_001.title, soup.title.text)
AssertionError: '첫 번째 포스트입니다.' not found in 'Blog'
-----
Ran 2 tests in 0.045s

FAILED (failures=1)
Destroying test database for alias 'default'...
```

- 오류 확인

```
base.html ×
1 <!DOCTYPE html>
2 {% load static %}
3 <html>
4
5 <head>
6     <title>{% block head_title %}Blog{% endblock %}</title>
7     <link rel="stylesheet" href="{% static 'blog/bootstrap/bootstrap.
```

```
<> post_detail.html ×
1 {% extends 'blog/base.html' %}
2
3 {% block head_title %}
4     {{ post.title }} - Blog
5 {% endblock %}
6
7 {% block main_area %}
```

피고인의 자백 - Blog

2. 메인영역 모듈화하기

□ post_detail.html 모듈화하기

✓ 포스트 영역과 댓글영역 구분하기

- 테스트 수행: post_area가 없다는 오류

```
=====
ERROR: test_post_detail (blog.tests.TestView.test_post_detail)
-----
Traceback (most recent call last):
  File "C:\work\python\mysite\blog\tests.py", line 76, in test_post_detail
    self.assertIn(post_001.title, post_area.text)
    ^^^^^^^^^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'text'
```

- 포스트 영역은 id="post-area" 인 <div>
- 댓글 영역은 id="comment-area" 인 <div>로 구분
- 테스트 후 결과 → 성공 확인

```
{% extends 'blog/base.html' %}

{% block head_title %}
    {{ post.title }} - Blog
{% endblock %}

{% block main_area %}
    <div id="post-area">

        <!-- Title -->
        <h1 class="mt-4">{{ post.title }}</h1>
        <h5 class="text_muted">{{ post.hook_text }}</h5>
        (... 생략 ...)
    {% endif %}

    <hr>
</div>

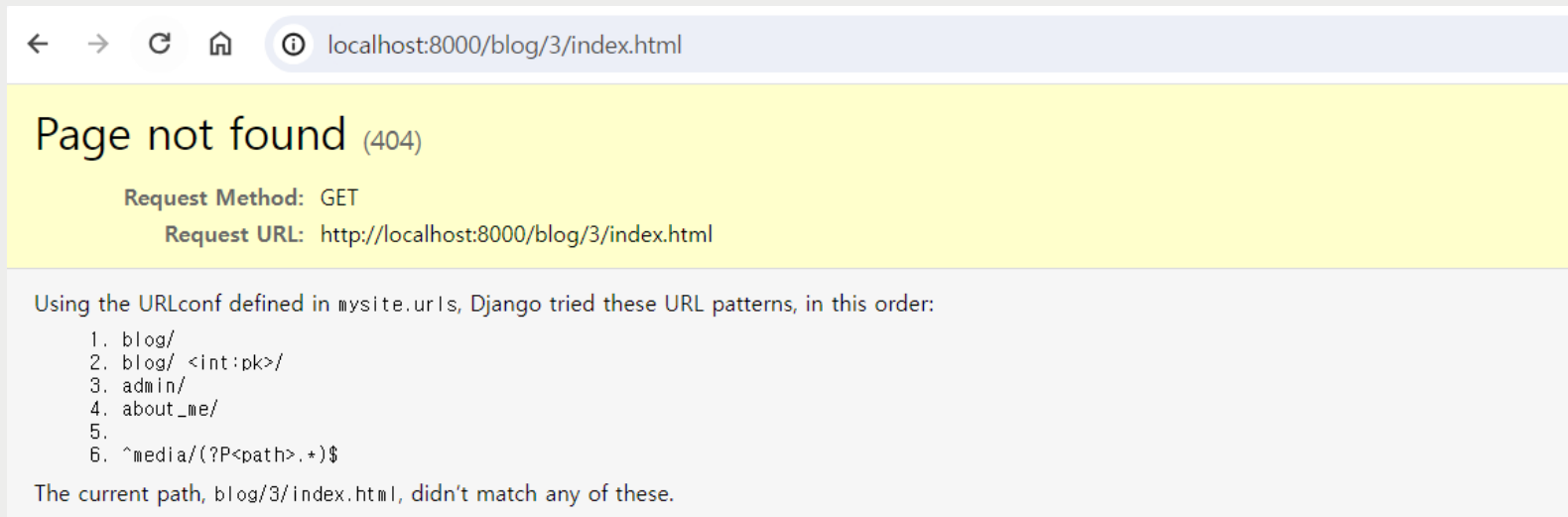
    <div id="comment-area">
        <!-- Comments Form -->
        <div class="card my-4">
            </div>
            (... 생략 ...)
        </div>
    </div>
{% endblock %}
```

3. 내비게이션바와 푸터 모듈화하기

□ 내비게이션 바 버튼에 링크 추가하기

✓ 현재 테스트 코드는 'Blog'와 'About me'라는 문구가 있는지만 확인

✓ 내비게이션 바 버튼 클릭하면 오류 발생



You're seeing this error because you have `DEBUG = True` in your Django settings file. Change that to `False`, and Django will display a standard 404 page.

3. 내비게이션바와 푸터 모듈화하기

□ 내비게이션 바 버튼에 링크 추가하기

✓ 테스트 코드 작성하기(내비게이션 바 테스트 모듈화)

- navbar_test()라는 별도의 함수로 생성 - 파싱된 HTML 요소에서 테스트하기 위해서 soup을 매개변수

```
def navbar_test(self, soup):
    navbar = soup.nav
    self.assertIn('Blog', navbar.text)
    self.assertIn('About Me', navbar.text)

def test_post_list(self):
    # 1.1 포스트 목록페이지를 가져온다.
    response = self.client.get("/blog/")

    # 1.2 정상적으로 페이지가 로드된다.
    self.assertEqual(response.status_code, 200)

    # 1.3 페이지 타이틀은 'blog'이다.
    soup = BeautifulSoup(response.content, "html.parser")
    self.assertEqual(soup.title.text, "Blog")

    # 1.4 내비게이션 바가 있다.
    navbar = soup.nav
    # 1.5 Blog, About Me라는 문구가 내비게이션 바에 있다.
    self.assertIn("Blog", navbar.text)
    self.assertIn("About Me", navbar.text)
    self.navbar_test(soup)
```

```
def test_post_detail(self):
    # 0. Post가 하나 있다.
    post_001 = Post.objects.create(
        title='첫번째 포스트입니다.',
        content='Hello World. We are the world.',
    )
    # 0.1 그 포스트의 url은 'blog/1/' 이다.
    self.assertEqual(post_001.get_absolute_url(), '/blog/1/')

    # 1. 첫 번째 post의 detail 페이지 테스트
    # 1.1 첫 번째 post url로 접근하면 정상적으로 작동한다. (s
    response = self.client.get(post_001.get_absolute_url())
    self.assertEqual(response.status_code, 200)
    soup = BeautifulSoup(response.content, 'html.parser')

    # 1.2 post_list 페이지와 똑같은 네비게이션 바가 있다.
    navbar = soup.nav # beautifulsoup를 이용하면 간단히 페이지
    self.assertIn('Blog', navbar.text)
    self.assertIn('About Me', navbar.text)
    self.navbar_test(soup)
```

3. 내비게이션바와 푸터 모듈화하기

□ 내비게이션 바 버튼에 링크 추가하기

✓ 내비게이션 바 버튼의 href 링크 테스트 코드 만들기

- <Do It Django> 또는 <Home>을 클릭 시 127.0.0.1:8000로 이동 - 대문 페이지
- <Blog>를 클릭하면 127.0.0.1:8000/blog로 이동, <About Me>는 127.0.0.1:8000/about_me로 이동

```
def navbar_test(self, soup):
    navbar = soup.nav
    self.assertIn('Blog', navbar.text)
    self.assertIn('About Me', navbar.text)

    logo_btn = navbar.find('a', text='Do It Django')
    self.assertEqual(logo_btn.attrs['href'], '/')

    home_btn = navbar.find('a', text='Home')
    self.assertEqual(home_btn.attrs['href'], '/')

    blog_btn = navbar.find('a', text='Blog')
    self.assertEqual(blog_btn.attrs['href'], '/blog/')

    about_me_btn = navbar.find('a', text='About Me')
    self.assertEqual(about_me_btn.attrs['href'], '/about_me/')
```

3. 내비게이션바와 푸터 모듈화하기

□ 내비게이션 바 버튼에 링크 추가하기

✓ 내비게이션 바 버튼의 href 링크 테스트 코드 만들기

- 테스트 수행: python manage.py test 하면 오류 발생
- base.html 수정: home의 태그 삭제

```
=====
FAIL: test_post_list (blog.tests.TestView.test_post_list)
-----
Traceback (most recent call last):
  File "C:\work\python\mysite\blog\tests.py", line 40, in test_post_list
    self.navbar_test(soup)
  File "C:\work\python\mysite\blog\tests.py", line 16, in navbar_test
    self.assertEqual(logs_btn.attrs['href'], '/')
AssertionError: './index.html' != '/'
- ./index.html
+ /
```

```
<a class="navbar-brand" href="/">Do It Django</a>
<button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNavDr
    aria-controls="navbarNavDropdown" aria-expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarNavDropdown">
    <ul class="navbar-nav">
        <li class="nav-item active">
            <a class="nav-link" href="/">Home<span class="sr-only">(current)</span></a>
        </li>
        <li class="nav-item">
            <a class="nav-link" href="/blog/">Blog</a>
        </li>
        <li class="nav-item">
            <a class="nav-link" href="/about_me/">About Me</a>
        </li>
        <li class="nav-item dropdown">
```

3. 내비게이션바와 푸터 모듈화하기

□ Include로 내비게이션 바와 푸터 모듈화 하기

✓ Include: 포함하고 싶은 템플릿(html) 정의

base.html

navbar.html

post_list.html

footer.html

실습 파일: blog/templates/blog/base.html

```
(...생략...)
<body>

{% include 'blog/navbar.html' %}

<div class="container my-3">
  <div class="row">
    (...생략...)
    {% include 'blog/footer.html' %}
  </div>
</div>
(...생략...)
```

실습 파일: blog/templates/blog/footer.html

```
<!-- Footer -->
<footer class="py-5 bg-dark">
  <div class="container">
    <p class="m-0 text-center text-white">
      Copyright &copy; Do It Django A to Z 2021
    </p>
  </div>
<!-- /.container -->
</footer>
```


3. 내비게이션바와 푸터 모듈화하기

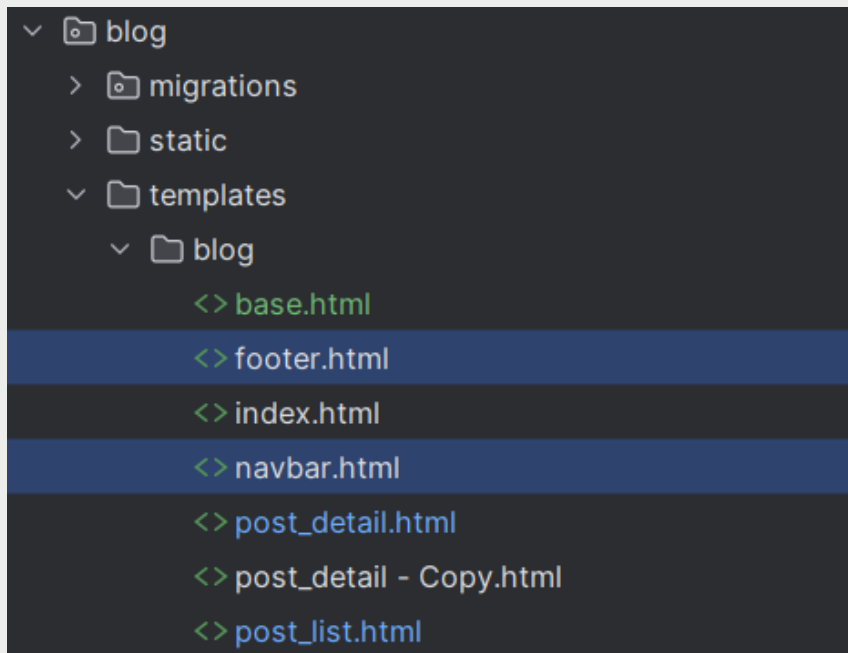
□ Include로 내비게이션 바와 푸터 모듈화 하기

✓ 내비게이션 바를 navbar.html로 모듈화 하기

- navbar.html을 만들고, base.html의 <nav> 부터 modal에 관련된 코드까지 잘라서 붙여 넣음

✓ 푸터를 footer.html로 모듈화 하기

- footer.html을 만들고, base.html의 <footer> 태그를 잘라서 붙여 넣음 → 푸터 내용 변경 가능



실습



- 메인영역 모듈화 해보기
 - ✓ post_list.html에 적용해보기
 - ✓ post_detail.html에 적용해보기
- 내비게이션 바와 푸터 모듈화 해보기

1. 다대일 관계(many to one)

□ A모델의 여러 레코드가 B 모델의 한 레코드에 연결 될 수 있는 관계

✓ 예시 1: Post 모델의 여러 post 레코드는 User 모델의 James에 연결 될 수 있음

→ James라는 사용자가 여러 개의 Post를 작성했음

✓ 예시 2: Post 모델의 여러 post 레코드는 Category 모델의 life style 태그와 연결됨



2. Foreign Key로 author필드 구현하기

❑ model.py에 author필드 구현하기

Post의 작성자를 다대일 구조로
User 모델과 연결

✓ on_delete: 연결되어 있던
User가 삭제될 때

- models.CASCADE
연결되어 있는 post 레코드까지
함께 삭제

실습 파일: blog/models.py

```
from django.db import models
from django.contrib.auth.models import User
import os

class Post(models.Model):
    title = models.CharField(max_length=30)
    hook_text = models.CharField(max_length=100, blank=True)
    content = models.TextField()

    head_image = models.ImageField(upload_to='blog/images/%Y/%m/%d/', blank=True)
    file_upload = models.FileField(upload_to='blog/files/%Y/%m/%d/', blank=True)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    author = models.ForeignKey(User, on_delete=models.CASCADE)

    def __str__(self):
        return f'[{self.pk}]{self.title} :: {self.author}'

(...생략...)
```

2. Foreign Key로 author필드 구현하기

❑ 마이그레이션 하기

✓ python manage.py makemigrations 수행하면 : author가 필수라서 오류 발생

```
C:\work\python\mysite (main -> origin)
(.venv) λ python manage.py makemigrations
It is impossible to add a non-nullable field 'author' to post without specifying a default. This is because the database
needs something to populate existing rows.
Please select a fix:
  1) Provide a one-off default now (will be set on all existing rows with a null value for this column)
  2) Quit and manually define a default value in models.py.
Select an option:
```

✓ 1) default 값을 지금 추가하기

✓ 2) 일단 취소하고 models.py 수정하기

▪ author = models.ForeignKey(User, null=True, on_delete=models.CASCADE)

```
  1) Provide a one-off default now (will be set on all existing rows with a null value for this column)
  2) Quit and manually define a default value in models.py.
Select an option: 1
Please enter the default value as valid Python.
The datetime and django.utils.timezone modules are available, so it is possible to provide e.g. timezone.now as
Type 'exit' to exit this prompt
>>> 1
Migrations for 'blog':
  blog\migrations\0004_post_author.py
  - Add field author to post
```

pk=1인 author로 지정

✓ python manage.py migrate

2. Foreign Key로 author필드 구현하기

❑ 마이그레이션 하기

✓ python manage.py runserver 실행 후 admin 페이지에 접근

Django administration WELCOME, PDW90 [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Blog > Posts > [3]피고인의 자백 :: pdw90

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

- Groups [+ Add](#)
- Users [+ Add](#)

BLOG

- Posts [+ Add](#)

Change post

[3]피고인의 자백 :: pdw90 HISTORY [VIEW ON SITE](#)

Title:

Hook text:

Content:

피고인의 자백이 고문·폭행·협박·구속의 부당한 장기화 또는 기망 기타의 방법에 의하여 자의로 진술된 것이 아니라고 인정될 때 또는 정식재판에 있어서 피고인의 자백이 그에게 불리한 유일한 증거일 때에는 이를 유죄의 증거로 삼거나 이를 이유로 처벌할 수 없다. 대통령의 임기는 5년으로 하며, 중임할 수 없다.

국무위원은 국무총리의 제청으로 대통령이 임명한다. 국가는 농지에 관하여 경자유전의 원칙이 달성될 수 있도록 노력하여야 하며, 농지의 소작제도는 금지된다. 국군은 국가의 안전보장과 국토방위의 신성한 의무를 수행함을 사명으로 하며, 그 정치적 중립성은 준수된다. 환경권의 내용과 행사에 관하여는 법률로 정한다.

Head image: 선택된 파일 없음

File upload: 선택된 파일 없음

Author:

2. Foreign Key로 author필드 구현하기

❑ 다른 사용자가 게시물을 작성할 때 테스트하기

✓ 새 사용자 생성

Django administration

Home > Authentication and Authorization > Users > Add user

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

- Groups + Add
- Users + Add**

BLOG

- Posts + Add

Add user

First, enter a username and password. Then, you'll be able to edit more user options.

Username:
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:
Your password can't be too similar to your other personal information.
Your password must contain at least 8 characters.
Your password can't be a commonly used password.
Your password can't be entirely numeric.

Password confirmation:
Enter the same password as before, for verification.

SAVE Save and add another Save and continue editing

2. Foreign Key로 author필드 구현하기

❑ 다른 사용자가 게시물을 작성할 때 테스트하기

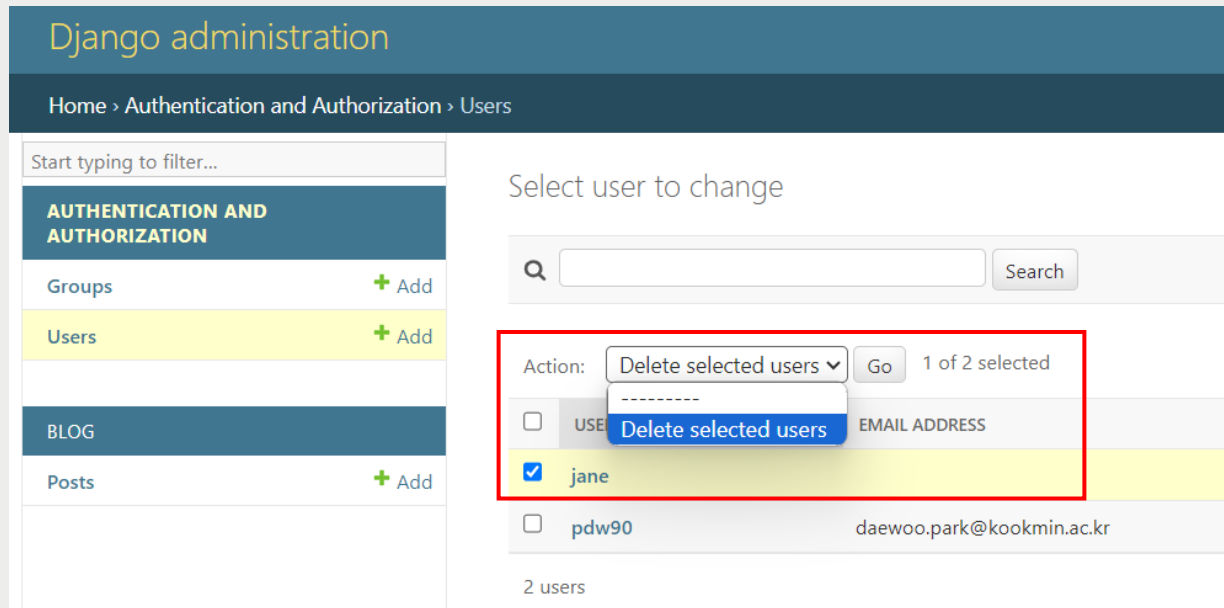
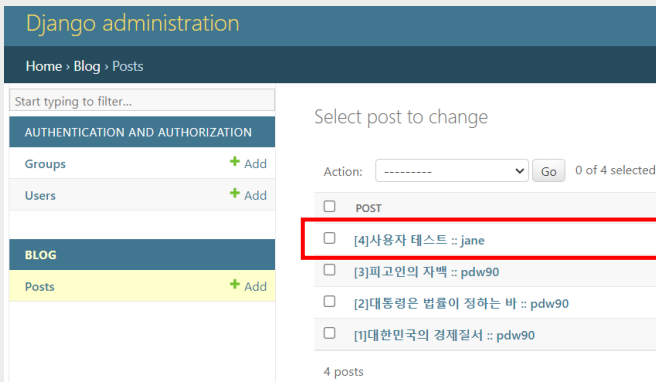
✓ 새로운 게시물 만들기

The screenshot shows the Django administration interface for a blog. The left sidebar contains a list of links: 'Groups', 'Users', and 'Posts'. The 'Posts' link is highlighted with a red box. The main content area shows the 'Add post' form. The form has fields for 'Title', 'Hook text', and 'Content'. Below these fields are 'Head image' and 'File upload' sections, each with a '파일 선택' (File select) button and a '선택된 파일 없음' (No file selected) message. The 'Author' field is a dropdown menu, and the 'jane' option is selected and highlighted with a red box. At the bottom of the form are three buttons: 'SAVE', 'Save and add another', and 'Save and continue editing'.

2. Foreign Key로 author필드 구현하기

❑ 다른 사용자가 게시물을 작성할 때 테스트하기

✓ 새로운 게시물 확인 및 작성자 삭제 후 게시물도 삭제되는지 확인[User, on_delete=models.CASCADE]



2. Foreign Key로 author필드 구현하기

❑ 다른 사용자가 게시물을 작성할 때 테스트하기

✓ 새로운 게시물 확인 및 작성자 삭제 후 게시물도 삭제되는지 확인[User, on_delete=models.CASCADE]

Django administration

Home > Authentication and Authorization > Users > Delete multiple objects

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

BLOG

Posts [+ Add](#)

Are you sure?

Are you sure you want to delete the selected user?

Summary

- Users: 1
- Posts: 1

Objects

- User: jane
 - Post: [4]사용자 테스트 :: jane

[Yes, I'm sure](#)

[No, take me back](#)

Select user to change



Search

Action: [-----](#) [Go](#)

0 of 1 selected



USERNAME

EMAIL ADDRESS



pdw90

daewoo.park@kookmin.ac.kr

Select post to change

Action: [-----](#) [Go](#)

0 of 3 selected



POST



[3]피고인의 자백 :: pdw90



[2]대통령은 법률이 정하는 바 :: pdw90



[1]대한민국의 경제질서 :: pdw90

3. 연결된 사용자가 삭제되면 빈 칸으로 두기

❑ on_delete=models.CASCADE → on_delete=models.SET_NULL

```
models.py ×
8     content = models.TextField()
9
10    head_image = models.ImageField(upload_to='blog/images/%Y/%m/%d/', blank=True)
11    file_upload = models.FileField(upload_to='blog/files/%Y/%m/%d/', blank=True)
12
13    created_at = models.DateTimeField(auto_now_add=True)
14    updated_at = models.DateTimeField(auto_now=True)
15
16    author = models.ForeignKey(User, on_delete=models.SET_NULL)
```

✓ 마이그레이션하면 오류 발생, on_delete=models.SET_NULL로 설정한 곳은 null 값도 허용해야 함

```
C:\work\python\mysite (main -> origin)
(.venv) λ python manage.py makemigrations
SystemCheckError: System check identified some issues:

ERRORS:
blog.Post.author: (fields.E320) Field specifies on_delete=SET_NULL, but cannot be null.
    HINT: Set null=True argument on the field, or change the on_delete rule.
```

3. 연결된 사용자가 삭제되면 빈 칸으로 두기

❑ `on_delete=models.CASCADE` → `on_delete=models.SET_NULL`

✓ 아래처럼 수정 후 마이그레이션 수행

실습 파일: blog/models.py

```
(...생략...)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

author = models.ForeignKey(User, null=True, on_delete=models.SET_NULL)
(...생략...)
```

null=True

- 해당 필드 값이 공란이어도 OK

on_delete: 연결되어 있던 User가 삭제될 때

- `models.SET_NULL`
해당 필드 값을 NULL (None)으로 바꿈

3. 연결된 사용자가 삭제되면 빈 칸으로 두기

❑ 포스트 작성자가 삭제되었을 때 테스트 하기

- ✓ obama라는 계정 생성
- ✓ obama로 포스트 생성
- ✓ obama라는 계정 삭제 후 포스트 목록 확인

Select post to change

Action: 0 of 5 selected

☐	POST
☐	[5]오바마 테스트2 :: obama
☐	[4]오바마 테스트1 :: obama
☐	[3]피고인의 자백 :: pdw90
☐	[2]대통령은 법률이 정하는 바 :: pdw90
☐	[1]대한민국의 경제질서 :: pdw90

Select post to change

Action: 0 of 5 selected

☐	POST
☐	[5]오바마 테스트2 :: None
☐	[4]오바마 테스트1 :: None
☐	[3]피고인의 자백 :: pdw90
☐	[2]대통령은 법률이 정하는 바 :: pdw90
☐	[1]대한민국의 경제질서 :: pdw90

5 posts

4. 포스트 목록페이지와 상세 페이지에 author 반영하기

□ 포스트 목록페이지에 작성자 추가 테스트 코드 작성

✓ User 생성 및 테스트 코드 추가(blog/test.py)

```
from django.contrib.auth.models import User

# pdw90 *
class TestView(TestCase):
    # pdw90 *
    def setUp(self):
        self.client = Client()
        self.user_trump = User.objects.create_user(username='trump', password='somepassword')
        self.user_obaba = User.objects.create_user(username='obama', password='somepassword')
```

```
# 3.1 게시물이 2개 있다면
post_001 = Post.objects.create(
    title="첫번째 포스트입니다.",
    content="Hello World. We are the world.",
    author=self.user_trump
)

post_002 = Post.objects.create(
    title="두번째 포스트입니다.",
    content="1등이 전부는 아니잖아요?",
    author=self.user_obaba
)
```

test_post_list 함수의 맨 아래쪽

```
self.assertIn(self.user_trump.username.upper(), main_area.text)
self.assertIn(self.user_obaba.username.upper(), main_area.text)
```

4. 포스트 목록페이지와 상세 페이지에 author 반영하기

□ 포스트 목록페이지에 작성자 추가 테스트 코드 작성

✓ 테스트 수행

```
=====
FAIL: test_post_list (blog.tests.TestView.test_post_list)
-----
Traceback (most recent call last):
  File "C:\work\python\mysite\blog\tests.py", line 79, in test_post_list
    self.assertIn(self.user_trump.username.upper(), main_area.text)
AssertionError: 'TRUMP' not found in '\nBlog\n\n\n\n두 번째 포스트입니다.\n1등이 전부는 아니잖아요?\nRead More →\n\n\nPosted on May 6, 2024, 3:34 p.m. by\nTBD \n\n\n\n첫번째 포스트입니다.\nHello Worl
d. We are the world.\nRead More →\n\n\n\nPosted on May 6, 2024, 3:34 p.m. by\nTBD \n\n\n\n\n\nOlder\n\n\nNewer →\n\n\n\n'
-----
Ran 2 tests in 1.382s
```

✓ post_list.html 수정 후 테스트 수행하고 웹 브라우저에서 확인

```
<div class="card-footer text-muted">
    Posted on {{ p.created_at }} by
    <a href="#"> {{ p.author | upper }} </a>
</div>
```

Read More →

Posted on March 26, 2024, 5:50 a.m. by PDW90

4. 포스트 목록페이지와 상세 페이지에 author 반영하기

□ 포스트 상세페이지에 작성자 추가 테스트 코드 작성

✓ 테스트 코드 작성

```
def test_post_detail(self):  
    # 0. Post가 하나 있다.  
    post_001 = Post.objects.create(  
        title='첫번째 포스트입니다.',  
        content='Hello World. We are the world.',  
        author=self.user_trump,  
    )
```

```
# 1.6 첫 번째 post의 content가 post-area에 있다.  
self.assertIn(post_001.content, post_area.text)
```

```
self.assertIn(self.user_trump.username.upper(), main_area.text)
```


□ 포스트 상세페이지에 작성자 추가 테스트 코드 작성

```
FAIL: test_post_detail (blog.tests.TestView.test_post_detail)
```

```
File "C:\work\python\mysite\blog\tests.py", line 115, in test_post_detail
```

```
AssertionError: 'TRUMP' not found in '\n\n첫번째 포스트입니다.\n\n\nby\n\n작성자명 쓸 위치(가변예정)\n\n\nPosted on May 6, 2024, 3:49 p.m.\n\n\nHello World. We are the world.\n\n\nLeave a Comment:\n\n\nit\n\n\nCommenter Name\n\nCras sit amet nibh libero, in gravida nulla. Nulla vel metus sceleris  
sollicitudin. Cras purus odio, vestibulum in vulputate at, tempus viverra turpis. Fusce condimentum nunc ac nis  
te fringilla. Donec lacinia congue felis in faucibus.\n\n\nCommenter Name\n\nCras sit amet
```

Posted on April 3, 2024, 8:51 p.m.

실습



- ❑ Foreign Key로 author필드 구현하기
 - ✓ models.CASCADE 와 models.SET_NULL 결과 확인해보기
- ❑ 포스트 목록페이지와 상세 페이지에 author 반영하고 테스트 해보기

감사합니다.